ЗАДАНИЕ (вставь строку ниже в поле ввода вместе с вложением):
Напиши ОДИН файл Python 3 (только стандартная библиотека) строго по этому ТЗ для ПК i5-3570 / 32 ГБ / Windows 10 22H2. Обязательны четыре этапа по порядку: --check (только чтение), --tune (обратимые изменения с --rollback), --fleet (подключение ПК в сеть агентов как рабочей машины, до 8 параллельных задач, только после первых двух), --selftest. Только готовый код целиком, все 12 критериев приёмки.

ПОЛНОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ: ПРОГРАММА-ИСПОЛНИТЕЛЬ ДЛЯ РАБОЧЕГО ПК (24/7, МАКСИМАЛЬНАЯ ЗАГРУЗКА)

==========

Источник: скриншот «О системе» заказчика (25.10.2026) + официальные спецификации Intel. Составлено до отправки в Батл.

1. ЖЕЛЕЗО (точно)
- Имя устройства: DESKTOP-LL7GE9
- Процессор: Intel Core i5-3570 @ 3.40 ГГц, 4 ядра, 4 потока (Hyper-Threading НЕТ), Ivy Bridge, 22 нм, L3 6 МБ, TDP 77 Вт, Turbo Boost 2.0 до 3.8 ГГц.
- Набор инструкций: x86-64, SSE4.1/4.2, AVX (первое поколение), F16C, AES-NI. ВАЖНО: AVX2 НЕТ, FMA3 НЕТ. Следствие: современные сборки Python 3.12+ с оптимизацией AVX2, часть библиотек машинного обучения, Node 20+ (при определённых сборках) могут требовать AVX2 — программа не должна на них опираться; браузеры Chromium работают, так как требуют только SSE3/AVX.
- Оперативная память: 32,0 ГБ.
- Видео: AMD Radeon R7 200 Series, 1004 МБ видеопамяти. Для вычислений и ИИ непригодна (ROCm не поддерживает, DirectML слабее CPU, 1 ГБ не вмещает даже 8B-модель в Q4).
- Накопители: SSD 232 ГБ KINGSTON + HDD 932 ГБ MAXTOR STM3160318AS (2009 г., ~100 МБ/с, случайный доступ медленный). Все профили браузера, кэш и очередь ОБЯЗАТЕЛЬНО на SSD; на HDD — только архив.
- Код устройства 3E31024C-5CCD-4GE5-3263-34025755C52C, код продукта 00331-20024-59828-AA009, x64.
- Электричество: простой 50–70 Вт, полная нагрузка 150–200 Вт. Круглосуточная нагрузка допустима, но программа должна уметь ограничивать себя по расписанию.

2. ОПЕРАЦИОННАЯ СИСТЕМА
- Windows 10 22H2 (поддержка официально закончилась 14.10.2025), x64, PowerShell 5.1, стандартные права администратора доступны.
- Известные особенности: на этом ПК стоят два диска, поэтому все пути кэша/профилей должны быть настраиваемыми и по умолчанию указывать на диск с наибольшей свободной скоростью (SSD).

3. ЧТО ДОЛЖНА ДЕЛАТЬ ПРОГРАММА (один файл pc_maxload.py, только стандартная библиотека Python 3.11+)
3.1 Канал задач: GitHub (приватный репозиторий Ayt1741/cloud-pc), файлы queue/tasks.json (очередь), state/pc-results.jsonl (результаты), state/pc-heartbeat.json (сердцебиение раз в 30 с). Доступ по HTTPS с токеном из файла, токен в логи не попадает.
3.2 Максимальная загрузка ПК (полноценный режим, не скрытный):
- пул параллельных воркеров, число задаётся --workers (по умолчанию 4, максимум по числу ядер);
- до 4 одновременных браузерных профилей (Playwright), каждый в отдельном подпроцессе;
- приоритет рабочих подпроцессов выше обычного (Windows: BELOW_NORMAL_PRIORITY_CLASS для служебных, NORMAL для рабочих), но с ограничением по памяти;
- если очередь пуста — выполнять полезную нагрузку: проверки диска (только чтение), пересчёт контрольных сумм архива, подготовка газетных текстов; простаивать запрещено;
- видимое окно консоли разрешено и предпочтительно (заказчику нужна максимальная отдача, а не скрытность).
3.3 Отказоустойчивость: watchdog перезапускает зависший подпроцесс (таймаут задачи), ограничение 3 попытки на задачу, при 3 падениях планировщика — самоперезапуск, лог в state/pc-log.txt с ротацией.
3.4 Дашборд: state/status.json каждые 10 с — загрузка ЦП %, свободная память МБ, число живых воркеров, длина очереди, число ошибок за час, время работы.
3.5 Режимы: --selftest (не менее 8 проверок PASS/FAIL), --once (одна итерация), --dry-run (показать, что сделал бы), --max-load N (ограничение в %), --workers N.
3.6 Автозапуск: установка задачи в Планировщик от SYSTEM с автоперезапуском при сбое, и отдельный скрипт удаления.

4. ЗАПРЕЩЕНО
- Выполнять произвольный код из сети, eval/exec над содержимым задач.
- Удалять файлы пользователя, трогать Defender, брандмауэр, UAC, Windows Update, подпись драйверов.
- Светить токен в логах и в выводе.
- Работать с чужими аккаунтами и обходить капчу.

5. КРИТЕРИИ ПРИЁМКИ (12)
1. python pc_maxload.py --selftest → не менее 8 PASS, код 0.
2. --dry-run не делает изменений и печатает план.
3. --once выполняет одну итерацию и корректно выходит.
4. Очередь читается и результаты пишутся в формате JSON Lines.
5. Токен не появляется в state/pc-log.txt (проверяется поиском).
6. При обрыве сети цикл не падает, а повторяет с задержкой.
7. Дашборд state/status.json обновляется и содержит загрузку ЦП и память.

8. Watchdog убивает зависшую задачу по таймауту и пишет причину.
9. Три падения планировщика приводят к самоперезапуску.
10. Пути кэша по умолчанию на диске с наибольшей свободной скоростью (SSD), настраиваются.
11. Установка автозапуска и удаление выполняются одним скриптом каждая.
12. Код без внешних зависимостей, кроме Playwright, который вызывается как внешняя команда.

ДОПОЛНЕНИЕ ОТ 25.10.2026 (обязательно, порядок этапов менять нельзя)
Программа должна работать в четыре этапа, строго по порядку:
ЭТАП 1 «--check» — только чтение: состояние ПК (процессор, ОЗУ, диски SSD/HDD, службы, сеть, автозапуск, версия Windows, наличие Python, свободное место). Ничего не меняет, пишет JSON и таблицу.
ЭТАП 2 «--tune» — оптимизация под круглосуточную работу: план питания, отключение лишних служб и автозапуска,
кэш и очередь на SSD, лимиты памяти на воркер, журнал изменений и «--rollback».
ЭТАП 3 «--fleet» — подключение ПК в сеть агентов как рабочей машины: регистрация self-hosted runner GitHub Actions,
запуск как служба, число параллельных задач по объёму ОЗУ (32 ГБ — до 8), перезапуск при падении, остановка «--fleet-stop». Только после успешных этапов 1 и 2 и только по явному флагу.
ЭТАП 4 «--selftest» — самопроверка всех режимов, не менее 8 строк, ничего не меняет.
Требования: только стандартная библиотека Python 3.8+, совместимость с i5-3570 (AVX есть, AVX2 нет), непригодность Radeon R7 200 для ИИ учитывается, все внешние вызовы с таймаутом.